

RunPod Serverless Case Study

FLUX.1-dev Image Generation with a ChatGPT-Style Web Interface

This project deploys the FLUX.1-dev text-to-image model on RunPod Serverless and exposes it through a polished chat-style web application. Users enter natural-language prompts and receive generated images as model responses, similar to conversational AI products such as ChatGPT or Claude.

Objective

Build a production-ready RunPod serverless endpoint capable of receiving prompt requests and returning generated images, then demonstrate the endpoint through a clean, responsive chat UI suitable for senior-level evaluation.

Architecture

- RunPod Serverless worker (handler.py) loads FLUX.1-dev on GPU and serves inference requests.
- Docker image packages CUDA runtime, Python dependencies, and pre-cached model weights.
- FastAPI backend proxies requests to RunPod and keeps API credentials server-side.
- React frontend provides a ChatGPT/Claude-style experience with conversation history.

Flow: Browser -> FastAPI -> RunPod endpoint -> GPU worker -> base64 PNG -> chat UI.

RunPod Deployment

- Built a CUDA 12.1 Docker image with FLUX.1-dev weights baked in for faster cold starts.
- Deployed as a queue-based serverless endpoint on a 24 GB+ GPU.
- Configured HF_TOKEN and HF_HOME environment variables for gated model access.
- Validated inference using scripts/test_endpoint.py and the RunPod console.

Application Features

- Chat-style prompt input with loading states and retry on failure.
- Conversation sidebar with local persistence.
- Generation settings: steps, guidance, resolution, seed.
- Download generated images directly from the UI.
- Mobile-responsive layout with sidebar navigation.

Technical Highlights

- Async job submission with status polling for long-running GPU inference.
- Model loaded once per worker instance to minimize per-request overhead.
- Clear separation between serverless worker, API layer, and frontend.
- Complete setup documentation for reproducible deployment.

Testing

The endpoint accepts JSON input with a required prompt field and optional generation parameters. Successful responses return a base64-encoded PNG image. The included CLI test script saves output.png, and the web app demonstrates end-to-end prompt-to-image flow.

Deliverables

- handler.py serverless worker
- Dockerfile and requirements
- RunPod deployment instructions
- ChatGPT-style React UI
- GitHub repository with setup guide

GitHub Repository

<https://github.com/germainsafari/Runpod>

Clone the repository, configure .env with RUNPOD_API_KEY, RUNPOD_ENDPOINT_ID, and HF_TOKEN, then follow README.md to build the worker image, deploy on RunPod, and launch the chat app.